ERHVERVSAKADEMI KØBENHAVN

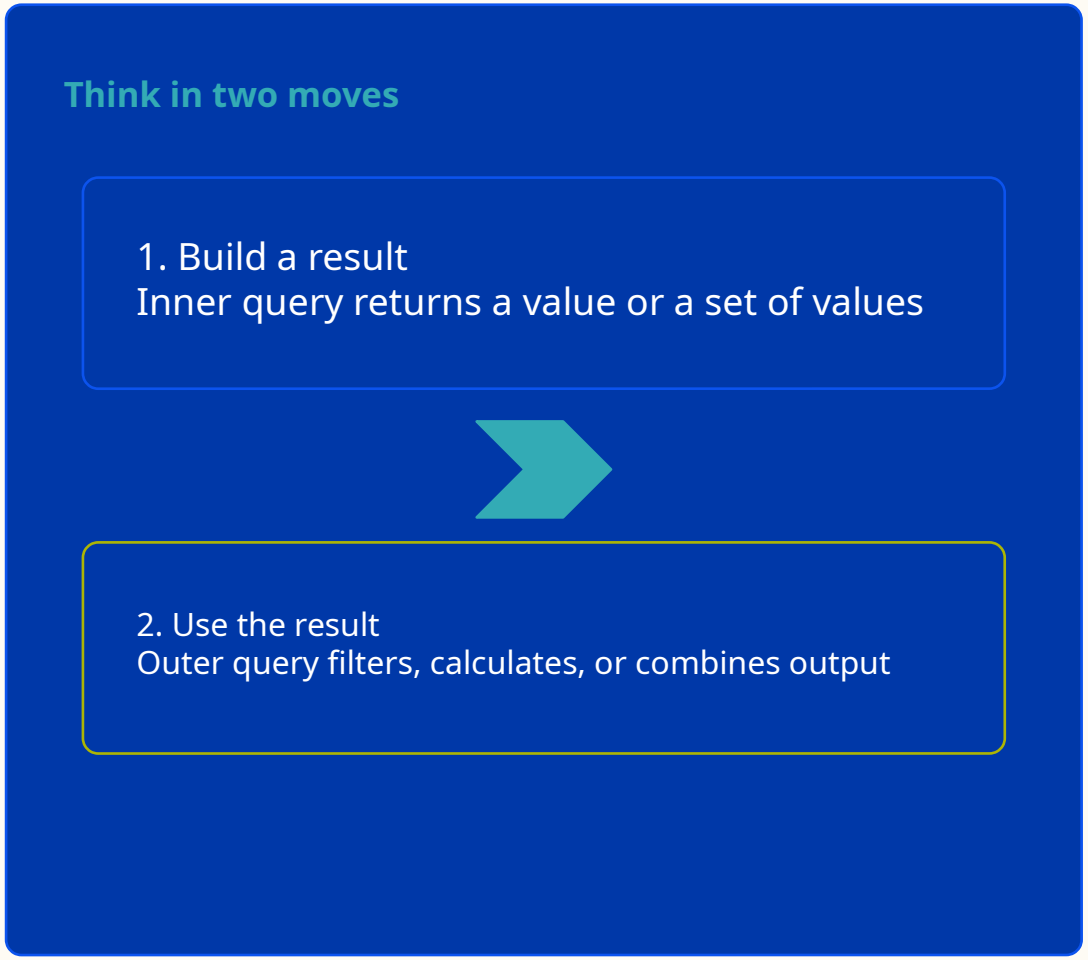
Complex queries

Subqueries • Derived tables • UNION

Dataset theme
Online shop: customers, archived_customers, products, orders

When one query is not enough

- Find products priced above the average
- Find customers who have placed an order
- Build totals first, then sort the totals
- Combine current and archived customer lists



Subqueries: the basic idea

Inner query

```
SELECT AVG(price)
FROM products;
```

1042.37

Run this part alone first



Outer query

```
SELECT product_name, price
FROM products
WHERE price > ( ... );
```

Rule

A subquery is a query used inside another query.
The inner result is used by the outer query.

Subqueries in WHERE: single value

Example

```
SELECT product_name, category, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
)
ORDER BY price DESC;
```

Use this pattern when

the inner query returns one value such as AVG, MAX, MIN, or COUNT

What the inner query returns

AVG(price) returns one number. That number can be used in the WHERE condition.

Read it as

Show products where price is greater than the average price.

Subqueries in WHERE: multiple values

Example

```
SELECT full_name, city, email
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

Use IN

IN is the safe choice when the inner query can return many rows.

Avoid this
customer_id = (SELECT customer_id FROM orders)

Reason

The inner query can return many customer_id values. = expects one value; IN accepts a set.

Derived tables in FROM

Example

```
SELECT customer_id, total_spent
FROM (
    SELECT customer_id, SUM(total_amount) AS total_spent
    FROM orders
    GROUP BY customer_id
) AS totals
ORDER BY total_spent DESC;
```

Step 1
Build totals per customer

Step 2
Query the temporary result

Think of it as
a temporary table created by a query and used immediately by the outer query

Calculated fields from subqueries

Example

```
SELECT product_name,
       price,
       (SELECT AVG(price) FROM products) AS avg_price,
       price - (SELECT AVG(price) FROM products) AS
diff_from_avg
FROM products;
```

Output idea

```
product_name    Smart Speaker
price           447
avg_price       1299.00
diff_from_avg   1042.37
                256.63
```

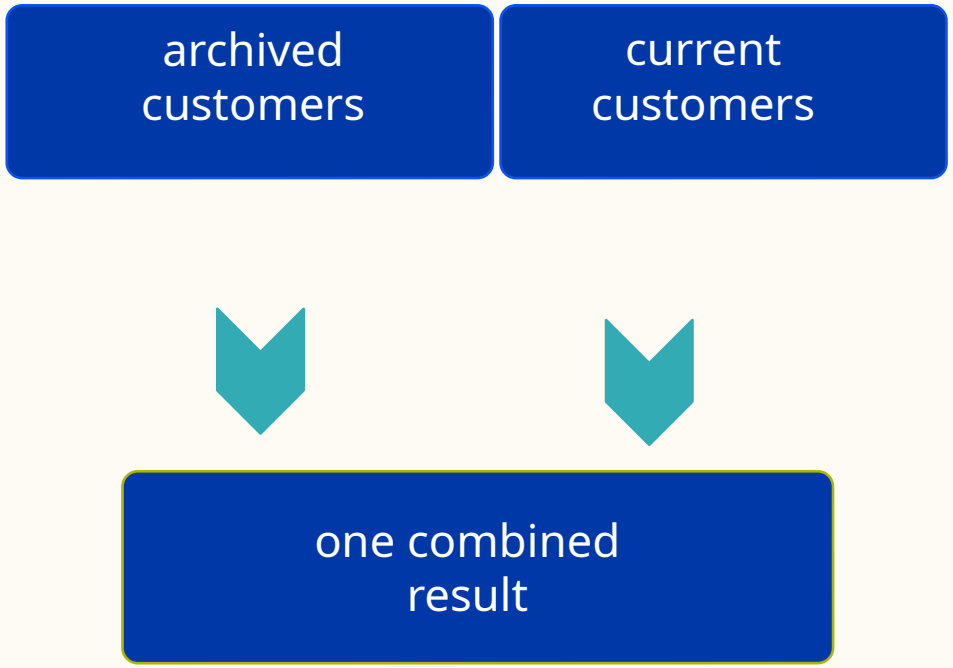
Use this when

a subquery provides a value that is useful as a column in the final result

Combining results with UNION

Example

```
SELECT full_name, city
FROM customers
UNION
SELECT full_name, city
FROM archived_customers;
```



UNION vs UNION ALL

UNION

Duplicates are removed

```
Asta Jensen
Emma Nielsen
Lucas Holm
```

UNION ALL

Duplicates are kept

```
Asta Jensen
Emma Nielsen
Emma Nielsen
Lucas Holm
```

Choose deliberately

Use UNION when duplicates should disappear. Use UNION ALL when every row matters.

Sorting combined results

Example

```
SELECT full_name, city
FROM customers
UNION ALL
SELECT full_name, city
FROM archived_customers
ORDER BY city, full_name;
```

Place ORDER BY at the end

The sort applies to the full combined result, not to each SELECT separately.

Sorted output idea

```
Aalborg   Alex Dahl
Aalborg   Mia Holm
Aarhus    Emma Nielsen
Aarhus    Emma Nielsen
```

Recap

Single-value subquery

One value from AVG, MAX, MIN, COUNT

Multi-row subquery

Use IN when the inner query returns many rows

Derived table

Build a temporary result in FROM

Calculated field

Use a subquery inside the SELECT list

UNION

Combine result sets and remove duplicates

UNION ALL + ORDER BY

Keep duplicates and sort the final combined output

PRACTICE

Practice: complex queries

Use the same dataset throughout the full exercise block.

- Subquery in WHERE using one value
- Subquery in WHERE using many values
- Derived table in FROM
- Calculated field from a subquery
- UNION, UNION ALL, duplicates, and sorting

Run the setup script session6_complex_queries_dataset.sql

Database session6_complex_queries

PRACTICE

Find products above the average price

Exercise 1

- Show product_name, category, and price
- Include only products with a price above the average product price
- Sort by price from highest to lowest

Tables

products

Focus

Build the query step by step. Run the inner query first when a subquery is involved. Keep the final output readable.

PRACTICE

Find customers with and without orders

Exercise 2

- Create one query that shows customers who have placed at least one order
- Create one query that shows customers who have not placed any orders
- Show full_name, city, and email

Tables

customers, orders

Focus

Build the query step by step. Run the inner query first when a subquery is involved. Keep the final output readable.

PRACTICE

Build totals in a derived table

Exercise 3

- Create a derived table that calculates total order amount per customer
- Show customer_id and total order amount
- Use the outer query to sort from highest to lowest total

Tables

orders

Focus

Build the query step by step. Run the inner query first when a subquery is involved. Keep the final output readable.

PRACTICE

Compare each product to the average

Exercise 4

- Show product_name and price
- Add the average product price as a separate column
- Add a calculated column that shows the difference between price and average price

Tables

products

Focus

Build the query step by step. Run the inner query first when a subquery is involved. Keep the final output readable.

PRACTICE

Combine current and archived customers

Exercise 5

- Combine full_name and city from customers and archived_customers
- Run the query once with UNION
- Run the query once with UNION ALL
- Sort the final result by city and full_name
- Observe the difference in duplicate handling

Tables

customers, archived_customers

Focus

Build the query step by step. Run the inner query first when a subquery is involved. Keep the final output readable.